Kutty Banerjee

710 Saratoga Avenue Apt#U304 San Jose, CA 95129
Phone: (925)209 7204 E-Mail: kutty.banerjee@gmail.com

Experience

System Software Architect, NVIDIA [2010 Sep ~ Present]

- Role involves leading cross functional efforts between CUDA/Graphics/hardware and OS software teams. Also involves hands on coding in individual projects.
- **Linux Kernel Page Table Coherence (Project Orion)**
  - Goal is to provide uniform address space between Denver(NVIDIA owned) CPU and GPU
  - My role is to scope out future GPU hw changes and drive Linux kernel VM changes for unified memory and demand paging support on the GPU with separate MMU units.
  - Doing performance analysis on current GPU to measure fault latency / TLB invalidation latency
  - Working closely with CUDA/HW teams to design low latency asynchronous GPU fault handling
- **Tegra T40 Memory Consistency Models**
  - Scoping out changes required in SW stack to implement Weak Consistency Model between Cortex A15 ARM CPU and Kepler GPU without explicit CPU/GPU L2 cache coherence.
  - Running experiments to evaluate if flushes/invalidates provide sufficient performance or if lack of GPU/CPU cache line coherence is a product killer. Measuring GPU L2 invalidate perf.
- **CUDA Page Based Memory Coherence (Upcoming Maxwell Architecture)**
  - Goal is to allow CUDA programs to pass pointers between CPU/GPUs using SW Page Based Cache Coherence and a weak memory model.
  - My role is to scope out GPU changes and **Linux kernel VM** changes for Page Base DSM.
  - Implemented proof of concept using MOESI in software to track page ownership.
  - Investigating ways to reduce false sharing. Studying real life app, memory traffic patterns.

Senior System Software Engineer, NVIDIA [2008 Sep ~ 2010 Aug]

- **Fermi Architecture Chip Lead (Resource Manager Team)**
  - Lead the overall technical efforts for Fermi Generation GPUs for the Resource Manager Team (OS Team) from functional simulation through RTL/emulation all the way to **Silicon bringup**
  - The role involved interacting with CUDA/OpenGL/DirectX and hw teams and driving technical implementation details for the Resource Manager team.
  - Also involved debugging very cross functional GPU hangs/bugs. Being familiar with 75% of the Fermi GPU, found myself become an ace debugger.
- **Power Aware Thread Scheduling (ELPG)** Designed changes to existing Thread Scheduler to allow engines to power down completely despite pending work. Tweaked to avoid this feature when latency sensitive work is pending. Worked with HW team to make state save/restore efficient.
- **Improve Verification of Asynchronous Concurrent Code** Implemented Probe Based Verification. Probes start up being noops in ISR code but can be dynamically modified at runtime to make it wait for certain events and thereby control order in which ISR sees certain events. Allows deterministically hitting/testing boundary conditions.
- **Vista Non stall interrupts and Lockless ISR** Worked closely with hw to design new interrupt tree for handling non stalling edge triggered interrupts. Rewrote a separate ISR that has minimal data dependency on the main ISR and rest of driver. Path uses extremely fine grained spin locks and lockless data structures
- **Windows 8 Mid Buffer Preemption** The goal here is to allow modification/reordering queued work and be able to inject new work with minimal latencies. Useful for desktop interactivity. My contribution was to provide APIs for Vista driver teams to use that allow submitted but not yet executed buffers to be reordered. Leverages Self Modifying Code Support from above.

System Software Engineer, NVIDIA **[2006 Sep ~ 2008 Aug]**

Software Lead (Fermi Thread Scheduling) Design and implement the GPU Thread Scheduling Code in driver for the Fermi Architecture. Identify perf pitfalls in previous architectures and drive HW/SW changes to fix them for the Fermi Architecture.

- **Improve Concurrency in Timeslice Scheduler** Rewrote scheduler to schedule CUDA/ Graphics / Video / DMA work streams between parallel HW instruction pipelines with the goal of saturating PCIE bandwidth, increasing locality and minimizing launch latency for timing sensitive work.
- **Implemented Pipelined Thread Scheduling** Previous generation thread scheduler was timeslice based whereas Vista pipelines work upfront. Matching the two led to stalls. Worked with HW to redesign GPU to suite this mode. Implemented a new Vista only Scheduling mode that avoids timeslicing and queues work upfront.
- **Fixed Perf Problems Thread Scheduling** Even though work is queued upfront by OS initial simulation showed hw was not taking advantage of this with better pipelining/prefetching. Worked with HW to add hints in driver to enable better prefetching / cache warming.
- **Efficient Scheduling Data Structures** GPU schedulable work is stored in a SW data structure called Runlist. Split runlist into separate runlists to avoid lock contention and accidental ordering of work between compute/graphics and DMA. Implemented indexed bitmap to get better CPU cache locality.
- **Implemented Instruction Emulation** Certain instructions on the GPU are emulated on the CPU. Implemented this layer and added features for out of order execution, bulk execution of methods to avoid repeated trap costs.
- **Self Modifying Code Support** Implemented API for OpenGL/CUDA/DX clients to use to modify their instruction stream after it has been queued in the GPU without introducing massive stalls. Under the hood this injects noops and acquire/releases and pipelined waits in the method stream before blitting in new code.
- **Fault Recovery** Worked closely with the hw team to design SW for recovery from page faults engine hangs and other conditions and identify changes required for new scheduling modes. Actual implementation was done by other members in my team.
- **Pipelined Page/Resource Swapping** Implemented framework to allow resources and virtual memory pages for contexts to be swapped in and out in a pipelined fashion to avoid work from getting stalled waiting for swap or wasting resources for threads that are not in the pipeline.

System Software Engineer, NVIDIA **[2005 June ~ 2006 Aug]**

<Joined the Resource Manager Team which implements the OS that manages the GPU. Its implemented partly in the host OS driver and parltly in microcode sitting inside the GPU>

- **G72 TurboCache** Helped the mmu component lead make changes in the Heap code to route surfaces to system memory. This was for a low end chip without local memory.
- Debugged and fixed bugs in MMU/Thread Scheduling/Power Management code. Did bringup.
- **Fermi NOMMU Support** Modified heap to allow physical and virtual addresses to match. This is very useful for debugging a new MMU hw.
- **Fermi MMU Page Table Support** Implemented initial code to support two level GPU page table. Added code to support single PDE pointing to two valid page tables. This was for Vista sysmem video memory swapping support.
- **Fermi MMU GPU TLB Invalidate** Wired up the TLB invalidate code. Optimized virtual address allocators to allow adjacent addresses to better utilize TLB using hints in PTE. HW had TLB coalesce/compress feature that aligned with this SW change.

- **GPU TLB invalidate Optimizations** Added code to batch invalidates to better use range based invalidation.
- **Fermi MMU NOINVALDATE mode** Modified allocators to skip TLB invalidates and L2 flushes entirely by avoiding virtual address reuse and clever alignment. This is useful in long RTL tests which allocate fixed number of surfaces. Resulted in 15% time reduction.
- **Fermi GPU Graphics Context Switch** Fermi added a new microcode engine to do this. Helped the component lead find areas to better pipeline save/load operations so they can happen in parallel. Implemented the initial version of context switch code. This was latter owned by a different engineer.

Education

MS Computer Science, Worcester Polytechnic Institute, MA [2003 Aug ~ 2005 May]

- **Thesis**: Remote Execution for 3D Graphics. A distributed 3D Rendering pipeline (RMESA) to offload floating point intensive operations to a cluster.
- Recipient of **WPI Institute Fellowship Award** for outstanding research work.

BS Computer Science, University of Mumbai [1999 July ~ 2003 May]

Skills

C/Java/x86 Assembly. Familiar with Linux/Solaris/Windows systems programming and Linux/Solaris kernel VM subsystems, dtrace. Asynchronous programming, threads, efficient locking, data structures.

Publications

- **A Middleware Architecture for Mobile 3D Graphics**
  Emmanuel Agu, Kutty Banerjee, Shirish Nilekar, Oleg Rekutin, Diane Kramer
  International Journal of Parallel, Emergent and Distributed Systems Volume 21, Issue 3, 2006
- **Estimating Mobile Memory Requirements and Rendering Time for Remote Execution of the Graphics Pipeline**
  Kutty Banerjee, Fan Wu, Emmanuel Agu - Proceedings of Eurographics 2005
- **Remote Execution for 3D Graphics on Mobile Devices**
  Kutty Banerjee, Emmanuel Agu
  International Conference on Wireless Networks, Communications and Mobile Computing, 2005
- **PowerSpy: Fine-Grained Software Energy Profiling for Mobile Devices**
  Kutty Banerjee, Emmanuel Agu
  International Conference on Wireless Networks, Communications and Mobile Computing, 2005
- **A Middleware Architecture for Mobile 3D Graphics**
  Emmanuel Agu, Kutty Banerjee, Shirish Nilekar, Oleg Rekutin, Diane Kramer
  Third International Workshop on Mobile Distributed Computing, 2006Side Projects
- **A Middleware Architecture for Mobile 3D Graphics**
  Emmanuel Agu, Kutty Banerjee, Shirish Nilekar, Oleg Rekutin, Diane Kramer

Side Projects (Java/C)

- **Fair spin lock**: Ticket locks on x86 implemented using lock add limit througput by disallowing all but one thread from contesting for lock. Modified this to allow a range of ticket owners to contest for lock instead of a single ticket owner.

- **Hierarchical Spinlock** (Distributed Spinlock) separate groups of threads use a node lock. Central lock is arbitrated between node locks. When local node has no contenders it parks it and other nodes can steal lock ownership using proxy threads. Reduces cacheline traffic substantially.
- **Concurrent Radix Tree** A modified radix tree with a modulo based Bloom Filter that uses optimistic loops around nodes instead of pessimistic locks.
- **Reader Lock** Allows readers to enter critical section without CAS or memory barriers. Writers protect page and force readers to fault and migrate to a separate page. Improves reader performance substantially.
- **Per CPU Page Tables** (work under progress) CAS instructions and fences on x86 can be expensive even for back to back CAS on same cacheline. Replaced CAS from critical section with regular store instructions. To prevent atomicity violation using Per CPU Page Tables with write protection for atomicity. Lock stealing done via IPIs.
- **Starvation Free Lockless Code** (work under progress) Optimistic CAS loops suffer from starvation because CAS tries are not wait-free. Added support to allow starved threads to force other contenders to fault-on-CAS to avoid starvation.

Visa Status US Permanent Resident